BME 133 Lab 3

Contributions: All three of us (Rohit, Neha, Dania) worked on each exercise together and discussed.

To begin, create a copy of this file using File -> Save as... and name the new file "Lab_1_LastName_FirstName.ipynb" using your last and first name. Make sure to execute all the code cells in the notebook and that the outputs show in your report. Before submitting, remember to add a collaboration statement in this mark down indicating the contribution of each member. You may edit the template below:

- Group member 1 first name, last name & student id: contributions here
- Group member 2 first name, last name & student id: contributions here
- if applicable, Group member 3 first name, last name & student id: contributions here

This lab will focus on Python functions. A typical non-trivial software or program will not only need to handle complex logical flows handled by the control flow statements we covered in the previous lab, but also some repeated calls to some things that need to be repeatedly done. If the repitition is immediate, we can use while or for loop control structures as discussed before. However, if we want to repeat something at a later time after doing something else, while and for loops are not going to be useful. Functions allow up to wrap up a packet of logic we want to execute often. We have implicitly been doing this often using the python built in function print. In this lab, we will learn how to define and call our custom functions, and learn about some of the other built-in python functions.

Defining and calling a function

Let's say you want to write your daily diary using python. Your morning routine probably looks the same. So you might want to wrap the morning routine as a python function. In Python, a function (without parameters) is defined using the *def* keyword, followed by a function name, paranthesis, then a colon. This constitutes the function's header. The thing that the function does, i.e. the function body, comes after in an indented block. For example, if we name our morning routine function as mouring_routine, the function definition and its body might be as follows.

In [32]:
```
def morning_routine(): # function header
    print("Logging mundane morning routine")
    print("   Wake up") # function body line 1
    print("   Wash face") # function body line 2, etc
    print("   Brush teeth")
```

```
    print("   Eat breakfast")
    print("End of morning routine log")
```

Execute the code above and see what happens. Did it print? No! The reason is that you have only defined the function so far. That is, you only declared what it does. You need to *call* the function for it to actually do what is stated in the function body. To call a function that has no paramters, you simply write the function name and follow it up with paranthesis. Execute the code below to call the morning_routine function.

In [33]:
```
morning_routine()
```

```
Logging mundane morning routine
   Wake up
   Wash face
   Brush teeth
   Eat breakfast
End of morning routine log
```

So now, our diary logging script might be something like the code that follows. Execute the block and see what happens.

In [34]:
```
print("Day 1")
print("----")
morning_routine() # First call to the function
print("Spend the day at school")
print("Eat dinner")
print("Sleep")
print("Day 2")
print("----")
morning_routine()
print("Spend the day at work")
print("Go to the gym")
print("Eat dinner")
print("Sleep")
```

```
Day 1
----
Logging mundane morning routine
   Wake up
   Wash face
   Brush teeth
   Eat breakfast
End of morning routine log
Spend the day at school
Eat dinner
Sleep
Day 2
----
Logging mundane morning routine
   Wake up
   Wash face
   Brush teeth
   Eat breakfast
End of morning routine log
Spend the day at work
Go to the gym
Eat dinner
Sleep
```

So the morning_routine function has been called twice. Without the function, you would have needed to repeat the code inside the function body every time you wanted to log the morning routine. Not the most exciting diary, but hopefully it illustrated the concept of defining and calling functions. Note that function body cannot be empty, if you need a placehoder for the function body, you may use the *pass* statement.

In [35]:
```
def difficult_function():
    pass # I will may be add the python code for what it does later.
```

Functions with arguments

When using functions in your programs, you often want to pass some information along to the function so that the function can process that information and produce the output you desire. Let us say you want to create a function that makes an appointment between a doctor and a patient. You will want to pass the doctor's and patient's information to the function. A function *parameter* allows you to define a function that takes some *argument* when the function is called. The parameters can be basic data types or the complex data structures we have discussed before. When definig a function with parameters, the parameters are written as comma separated names inside the paranthesis. For instance, the function to make an appointment between a doctor and patient might look like:

In [36]:
```
def make_appointment(doctor, patient):
    print("Making an appointment between " + doctor + " and " + patient)
```

When calling a function, you need to pass *arguments* to the function that will be assign values to the parameters in the function definition. Arguments are passed to the function as a comma separated list inside the paranthesis during the function call. For instance you can pass the specific doctor's and patient's information as arguments as follows.

In [37]:
```
doctor1 = "House"
doctor2 = "Drake Ramoray"
patient1 = "John"
patient2 = "Jane"

make_appointment(doctor1, patient1)
make_appointment(doctor2, patient2)
```

```
Making an appointment between House and John
Making an appointment between Drake Ramoray and Jane
```

Often, the code that called a function wants the function to return some value after processing the data that the caller passed into it. The Python keyword *return* allows us to do that exactly as illustrated with the celsius to fahrenheit convertor below.

In [38]:
```
def convert_celsius_to_fahrenheit(celsius):
    fahrenheit = celsius * 9 / 5 + 32
    return fahrenheit
```

In [39]:
```
print("35 C is " + str(convert_celsius_to_fahrenheit(35)) +  " F")
```

```
35 C is 95.0 F
```

It is important to note that variables inside a function body have a *local scope*. That means they cannot be directly accessed outside of the function. Try executing the code below and see what happens.

In [40]:
```
convert_celsius_to_fahrenheit(35)
print(fahrenheit)
```

```
---------------------------------------------------------------------------
NameError                                 Traceback (most recent call last)
Cell In[40], line 2
      1 convert_celsius_to_fahrenheit(35)
----> 2 print(fahrenheit)

NameError: name 'fahrenheit' is not defined
```

You should get an error saying the name is not defined. That is because the variable fahrenheit only lives inside the body of the convert_celsius_to_fahrenheit function. After the function is executed and returns a value, the code execution goes back to the caller and only the value returned remains and the *local variables* of the function disappear. This is intentional and it is to protect the local variables from being modified. We can fix the previous issue by assigning the value returned by the function to a new variable as follows.

In [ ]:
```
fahrenheit_of_35 = convert_celsius_to_fahrenheit(35)
print(fahrenheit_of_35)
```

If you want a variable to have a *global* scope, you may put the *global* keyword in front of it as shown below. But this is generally considered bad practice, so use with caution.

In [ ]:
```
def global_test():
    global x
    print(x)

x = 5
global_test()
```

Built-in functions

You have already encountered one Python built-in function, the *print* function. You can see the parameters of the print function definition at https://docs.python.org/3/library/functions.html#print. It looks like print(*objects, sep=' ', end='\n', file=None, flush=False). It looks familiar, with the *objects*, *sep*, *end*, *file* and *flush* parameters. But it has a few things that look a bit different. The star in front of objects is to indicate it correponds to a variable number of parameters, i.e. you can pass multiple arguments to be assigned to objects. Suppose for a second that *objects is the only parameter of print and consider the following code:

In [ ]:
```
print("hello", "world")
```

As you suspect, it printed the phrase "hello world." What happened was that the arguments "hello" and "world" were bundled as a tuple and assigned to the objects parameter. So the objects parameter is able to hold multiple arguments during the function call. It can also take more arguments as shown in the code below.

In [ ]:
```
print("hello", "world", "from", "the", "US")
```

How about the other paramters like sep? As you might have guessed, it means that when the print is called, use sep as the separator. Since *objects can consume a variable number of arguments, the next parameter must be called using a key value syntax. The equal sign in the function header indicates sep has a default value of ' ', which is space. That is why in the previous code cells, hello and world were separated by space. Execute the code below to see what happens.

In [ ]:
```
print("hello", "world", sep = '_')
```

As you can see now hello and world are separated by an underscore. You may also call print with the additional paramters. For instace, to end the print statement with a full

stop and a space instead of a new line ("\n" tells the print statement to end with a new line), you can pass the key value pair for end as in the third line of the following code.

In [ ]:
```
print("Hello world that will endd in a new line")
print("This will print in a new line")
print("Hello world ended with a period and space", end=". ")
print("This is the next statement but comes after a space instead of a new l
```

Python also has a function called *input* to get input from the user. Execute the code below to see how it operates. In Jupyter notebooks in VSCode, the input is entered at the top where you typically choose the kernel.

In [ ]:
```
print('Enter your name:')
yourname = input()
print('Hello, ' + yourname)
```

Python has many additional built in functions that do other things apart from input and output. The following code illustrates some of the math functions. Execute the code and guess what each of the functions do.

In [ ]:
```
print(min(5,2,3))
print(max(5,2,3))
print(abs(-10))
print(pow(3,2))
```

Recursion

Functions can call other functions. Recursive functions self-reference, that is, they call themselves.

An interesting conceptual references to recursion comes with describing how to open nesting dolls. You can create a function called open_doll(). Given a stack of dolls them, you would recursively open them, calling open_doll() until you've reached the inner most doll.

Let's look at an example. Suppose you have n doctors you want to assign to n patients where each patient gets exactly one doctor. The number of unique ways you can assign a doctor to a patient is given by the factorial function. For a postive integer n greater than or equal to 1, n factorial, denoted n! is defined to be n! = n*(n-1)*(n-2)...1. For instance 2! = 2*1 = 2. 3! = 3*2*1 = 6, and so on. So if you have two doctors House and Drake, and two patients John and Jane, you may assign House to John and Drake to Jane, or House to Jane and Drake to John. So there are 2 possiblities, i.e. 2!.

It follows from the defition of factorial that n! =n*(n-1)!. This recursive relation is easy to implement as a recursive function. A Python function for computing the factorial is given below. Execute the code and see what happens.

In [ ]:
```python
def factorial(i):
    if i == 1:
        return 1
    else:
        return i*factorial(i-1)

print(factorial(3))
print(factorial(10))

# i = 3
```

```
6
3628800
```

You can see the factorial function quickly grows! When we write recursive functions, it is important to have a base case, like the factorial of one being one, so that the recursion stops. Otherwise the function can keep calling itself an infinite number of times, or more realistically until the memory runs out.

Exercises

Exercise 0

Create and call a function with zero parameters that takes a patient's name as an input from the terminal (see input function discussion in the built-in functions section) and greets them by printing their name.

Exercise 1

Create a new function to make appointment between a patient and a doctor. The function should have four paramters: the patients name, a list of intergers corresponding to the patient's available hours, the doctor's name and a list of integers corresponding to the doctor's available times. For simplicity the available hours are a subset of {9,10,11,12,13,14,15,16,17}, i.e. 9 to 5 in military time and that the appointment blocks are an hour long. In the function body, check if the patient and doctor have a common hour. If they do, print a statement saying "Booking at time x between patient p and doctor d", where x is the common hour, p is the patient name and d is the doctor's name. After printing, return 0 to indicate success. Otherwise print a statement saying "Common time is not available for patient p and doctor d", where p is the patient's name and d is the doctor's name. Also return a value of 1 indicating failure. (Hint: you might need to loop through the available times of both.) Test your function by calling the function once with a patient and a doctor which have a common time and another time by calling the function with a patient and a doctor that do NOT have a common time.

In [ ]:
```python
# Your code here
```

```
def greet_patient():
    patient_name = input('What is your name?')
    # return print("Hello" + " " + patient_name)
    return print(f"Hello {patient_name}")

greet_patient()
```

```
Hello welcome to our clinic Rohit
```

In [ ]:
```
# Create a func with four params (patient's name, patient avail hours, doc's
# Check if patient and doc have a common hour
# If TRUE print "Booking...at time x between patient p and doctor d" and ret
# ELSE print "common time..." and return 1

def make_appointment(patient_name, patient_hours, doc_name, doc_hours):
    common_hour = None

    for pat_hour in patient_hours:
        for doc_hour in doc_hours:
            if pat_hour == doc_hour:
                common_hour = pat_hour

    if common_hour:
        print(f"Booking at time {common_hour} between patient {patient_name}
    else:
        print(f"Common time is not available for patient {patient_name} and



make_appointment('Dania', [9,10,11,12,13,14], 'Dr.Pepper', [14, 15, 16, 17])
make_appointment('Dania', [9,10,11,12,13,14], 'Dr.Pepper', [15, 16, 17])
```

```
Booking at time 14 between patient Dania and doctor Dr.Pepper.
Common time is not available for patient Dania and doctor Dr.Pepper
```

Exercise 2

Bacteria often grow by dividing into two. Suppose each new generation of bacteria divides at the same time. The number of bacteria after the nth round is B(n) = 2*B(n-1), where B(1) = 1. Implement a recursive function to determine the number of bacteria after n rounds. Test your function to determine the bacterial population after 10 rounds.

In [41]:
```
def bacteria_growth(n):

    if n == 1:
        return 1
    else:
        return 2 * bacteria_growth(n-1)

bacteria_growth(10)
```

Out[41]: 512